This flag disables RSpec's original syntax:

09-configuring-rspec/05/rspec_configure.rb

```
# Old syntax

describe SwissArmyKnife do  # bare `describe` method
  it 'is useful' do
    knife.should be_useful  # `should` expectation
  end
end
```

...in favor of the style in RSpec 3:

09-configuring-rspec/05/rspec_configure.rb

```
# New syntax

RSpec.describe SwissArmyKnife do  # `describe` called on the `RSpec` module
  it 'is useful' do
    expect(knife).to be_useful    # `expect()`-style expectation
  end
end
```

The old syntax relied heavily on monkey-patching core Ruby objects. The new zero-monkey-patching mode does not. The result is fewer gotchas and edge cases in your specs. Moreover, your code will continue to work with future versions of RSpec. For more information about this mode, see the RSpec blog post that introduced it.[14]

Random Order

As we discussed in *Testing the Invalid Case*, on page 89, we recommend you configure RSpec to run your specs in random order:

09-configuring-rspec/05/rspec_configure.rb

```
RSpec.configure do |config|
  config.order = :random
end
```

Running your specs in random order helps surface ordering dependencies between your examples. You'll be more likely to discover these problems when they first appear, and you'll be able to fix the bug while the code is still fresh in your mind.

Adding Your Own Settings

All of the settings we've seen so far ship with RSpec. But you're not limited to those. RSpec provides an API for adding new settings, which you can then use in your own libraries that extend RSpec's behavior.

14. http://rspec.info/blog/2013/07/the-plan-for-rspec-3/#zero-monkey-patching-mode